Day 15: Search, Filter and Disk Space Utilities

grep, cat, sort, uniq, find, df and du: simple notes with examples and commands

1. Why Search and Filter Data?

- **Saves time:** find the one line or file you need among thousands.
- **Handles big data:** logs and folders can be huge, so reading by eye is impossible.
- **Used everywhere:** in scripts, cron jobs and system administration.

Term	Meaning in simple words
Pattern	The word or text you are looking for.
Filter	Keep only the lines you want and drop the rest.
Case-sensitive	'Error' and 'error' are treated as different words.
Recursive	Go inside a folder and all its sub-folders too.
Wildcard *	Matches any text. `*.txt` = every file ending with .txt
Duplicate	A line that appears more than once.
Pipe |	Sends the output of one command into the next command.

The pipe: connecting commands

The pipe symbol **|** works like a pipeline: the result of the left command becomes the input of the right command. This lets you build powerful commands from small tools.

```
cat numbers.txt | sort -n      # cat output goes into sort
```

2. grep: Search for a Pattern

grep (**G**lobal **R**egular **E**xpression **P**rint) searches files or input for lines that contain a pattern and prints those lines.

Syntax: `grep [OPTIONS] PATTERN [FILE]`

Option	Meaning
`-i`	**I**gnore case: 'error', 'Error' and 'ERROR' all match.
`-v`	In**v**ert: show lines that do **not** match.
`-n`	Show line **n**umbers.
`-r`	**R**ecursive: search all files inside a folder.
`-c`	(extra) **C**ount the matching lines.

```
# Find lines containing "error" in a log file
grep 'error' system.log
```

Remember: grep is case-sensitive by default. 'error' will not match 'ERROR'. Add **-i** to match both.

3. cat: Show File Content

cat (con**cat**enate) reads a file and displays it on the screen. It can also join several files.

Syntax: `cat [OPTIONS] [FILE]`

```
cat example.txt              # show a file
cat -n example.txt           # show with line numbers
cat a.txt b.txt > all.txt    # join two files into a new file
```

Tip: For very long files use `less filename` (press q to quit) or `head` / `tail`, because cat prints everything at once.

4. sort: Arrange Lines in Order

sort arranges lines of a file or input in order. By default it sorts like a dictionary (alphabetical).

Syntax: `sort [OPTIONS] [FILE]`

Option	Meaning
-r	**R**everse the order.
-n	Sort **n**umerically (by number value).
-k N	Sort by **k**ey: the N-th column (field) of each line. (Homework topic)
-u	(extra) Remove duplicates while sorting.

```
# Sort a list of numbers
cat numbers.txt | sort -n
```

Why -n is needed

numbers.txt	sort	sort -n	sort -nr
10, 2, 33, 4, 1	1, 10, 2, 33, 4	1, 2, 4, 10, 33	33, 10, 4, 2, 1

Without **-n**, '10' comes before '2' because it compares character by character, like words.

Homework: sort by a column with -k

```
# marks.txt has two columns:  name  marks
sort -k2 -n marks.txt        # sort by column 2 (marks), small to big
sort -k2 -nr marks.txt       # biggest marks first
sort -k1 marks.txt           # sort by name (column 1)
```

5. uniq: Remove Repeated Lines

uniq removes duplicate lines, but only when the duplicates are **next to each other**. So always **sort first**.

Syntax: `uniq [OPTIONS] [FILE]`

Option	Meaning
`-c`	**C**ount how many times each line appears.
`-d`	Show only **d**uplicated lines.
`-u`	(extra) Show only lines that appear once.

```
# Unique lines of a file
sort input.txt | uniq

# Count each name, most frequent first (very useful!)
sort names.txt | uniq -c | sort -rn
```

Common mistake: uniq alone does not sort. If duplicates are far apart, uniq will not remove them. Use `sort file | uniq` every time.

6. find: Locate Files and Folders

find searches the folder tree for files and folders that match your conditions: name, type, size or time.

Syntax: `find [PATH] [OPTIONS] [EXPRESSION]`

Option	Meaning	Example
`-name`	Search by name (case-sensitive)	-name "*.txt"
`-iname`	Search by name (ignore case)	-iname "report*"
`-type`	f = file, d = directory	-type f
`-size`	By size: + bigger, - smaller. Units: k, M, G	-size +100k
`-mtime`	By modified time in days: - within, + older than	-mtime -7

PATH is where to start: `.` = current folder, `/home` = home folders, `/var/log` = system logs. Use quotes around patterns with * so the shell does not change them.

```
# All .txt files in the current folder (and below)
find . -name "*.txt"

# Files larger than 1 MB
find /home -size +1M

# Files modified in the last 7 days
find /var/log -mtime -7
```

Understanding -mtime and -size

Write	Meaning
`-mtime -1`	Modified within the last 1 day
`-mtime -7`	Modified within the last 7 days
`-mtime +7`	Modified **more than** 7 days ago (old files)
`-size +100k`	Bigger than 100 KB
`-size -10M`	Smaller than 10 MB

Homework: -type and combining conditions

```
find . -type f                    # only files
find . -type d                    # only directories (folders)
find . -type f -name "*.log"      # files ending in .log
find /var/log -type f -size +1M   # big log files
```

Remember: Conditions written together must **all** be true. Searching system folders like /var/log as a normal user shows 'Permission denied' messages: use `sudo find ...`, or hide them with `find ... 2>/dev/null`.

7. Bonus Commands: Check Disk Space (df and du)

Servers can run out of disk space, and then backups, logs and cron jobs fail. Two commands help you check space quickly: **df** and **du**.

Command	Full form	Answers the question
df	**d**isk **f**ree	How full is each disk or partition?
du	**d**isk **u**sage	How much space does this folder or file use?

Important: df -sh is a common mix-up: **df** has no **-s** option and shows an error. The correct commands are **df -h** (whole disk) and **du -sh** (one folder). You can combine options: **-sh** means **-s** and **-h** together.

df: how full are my disks?

Syntax: `df [OPTIONS] [PATH]`

Option	Meaning
`-h`	**H**uman-readable: show sizes as K, M, G instead of raw blocks.
`-T`	Show the file system **T**ype (ext4, xfs, tmpfs).
`-i`	(extra) Show inode usage (inodes were explained in Day 12).

```
df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        50G   20G   28G  42% /
/dev/sda2       100G   60G   35G  64% /home
tmpfs           2.0G     0  2.0G   0% /dev/shm

df -h /home            # only the disk that holds /home
```

Column	Meaning in simple words
Filesystem	The disk or partition name.
Size	Total size.
Used / Avail	Space already used / space still free.
Use%	Percentage full. Above 90% needs attention!
Mounted on	The folder where this disk is attached, for example / or /home.

du: how big is this folder?

Syntax: `du [OPTIONS] [PATH]`

Option	Meaning
`-h`	Human-readable sizes (K, M, G).
`-s`	**S**ummary: show only one total for the folder, not every sub-folder.
`--max-depth=1`	Show the folder and its first-level sub-folders only.
`-a`	(extra) Show files as well as folders.

```
du -sh ~/Documents
350M    /home/student/Documents          # one total

du -sh *                               # size of each item in this folder
4.0K    notes.txt
120M    projects

du -h --max-depth=1 ~                  # size of each sub-folder
```

Find the biggest folders (combine with sort and head)

```
du -h --max-depth=1 /var | sort -rh | head -5
```

Part	Meaning
`du -h --max-depth=1 /var`	Size of every first-level folder inside /var.
`| sort -rh`	Sort them **r**everse (biggest first). **-h** understands K, M, G, so 2G ranks above 500M.
`| head -5`	Show only the first 5 lines, the 5 biggest.

Remember: Use **sort -h** (not -n) for du -h output. With -n, '9K' would wrongly rank above '2G'.

Real-life task: the disk is full, what now?

Step	Command	Purpose
1	df -h	See which disk is nearly full (Use% high).
2	sudo du -h --max-depth=1 / 2>/dev/null | sort -rh | head	Find the biggest top-level folders.
3	sudo find /var -type f -size +500M	List very big files in that folder.
4	sudo gzip big.log or sudo rm old.log	Compress or delete files you no longer need.

Careful: Never delete system files in a hurry. Compress or move large old logs and backups first (see Day 14: tar and gzip), and use a cron job to clean old backups automatically.

8. Step-by-Step Practice Lab

Task: Goal: Create sample files and try every command safely in your own home folder.

Step 1: Create a practice folder with a sample log

The heredoc (<< 'EOF') writes everything up to the word EOF into the file.

```
mkdir ~/search_lab && cd ~/search_lab
cat > system.log << 'EOF'
Oct 10 10:00:01 INFO Server started
Oct 10 10:05:22 ERROR Disk full
Oct 10 10:07:45 WARNING High memory usage
Oct 10 10:10:03 error Connection lost
Oct 10 10:15:30 INFO User login
Oct 10 10:20:11 ERROR Disk full
EOF
```

Step 2: Create the other sample files

```
printf "10\n2\n33\n4\n1\n" > numbers.txt
printf "Ravi\nAnita\nRavi\nSourabh\nAnita\nRavi\n" > names.txt
printf "Ravi 75\nAnita 9\nSourabh 100\nMeena 82\n" > marks.txt
```

Step 3: Practice grep

```
grep 'error' system.log
Oct 10 10:10:03 error Connection lost

grep -i 'error' system.log          # 3 lines: ERROR, error, ERROR
grep -in 'error' system.log         # same, with line numbers 2: 4: 6:
grep -v 'INFO' system.log           # everything except INFO lines
grep -ic 'error' system.log
3
```

Step 4: Practice sort and uniq

```
sort -n numbers.txt
1
2
4
10
33

sort -k2 -nr marks.txt
Sourabh 100
Meena 82
Ravi 75
Anita 9

sort names.txt | uniq
Anita
Ravi
Sourabh

sort names.txt | uniq -c | sort -rn
      3 Ravi
      2 Anita
      1 Sourabh
```

Step 5: Create files for find, then search

The **touch -d** command creates a file with an old date, and **dd** makes a 2 MB file.

```
mkdir logs
touch logs/app.log logs/sys.log notes.txt report.txt
touch -d "10 days ago" logs/old.log
dd if=/dev/zero of=big.dat bs=1M count=2 2>/dev/null

find . -name "*.txt"                 # notes, report, names, ...
find . -iname "REPORT*"              # ./report.txt
find . -type d                       # . and ./logs
find . -size +1M                     # ./big.dat
find . -name "*.log" -mtime -2       # app.log and sys.log
find . -name "*.log" -mtime +7       # ./logs/old.log
```

Step 6: Check disk space

du -sh shows the total size of the lab folder (it includes the 2 MB big.dat). Exact sizes differ on your system.

```
df -h .                       # space on the disk holding this folder
du -sh ~/search_lab           # total size of the folder
2.1M    /home/student/search_lab
du -sh *                      # size of each item
2.0M    big.dat
du -h --max-depth=1 | sort -rh | head -3
```

Step 7: Clean up

```
cd ~ && rm -r ~/search_lab
```

9. Practical Exercises (from class)

Exercise 1: Search for all error logs in /var/log

```
sudo grep -r 'error' /var/log
sudo grep -ri 'error' /var/log      # also match Error and ERROR
```

-r goes through every file in the folder. System logs need **sudo**. For a large result, add `| less` or `| head`.

Exercise 2: List all .log files modified in the last 2 days

```
find /var/log -name "*.log" -mtime -2
```

Exercise 3: Sort a file of names alphabetically and remove duplicates

```
sort names.txt | uniq
sort -u names.txt                # shortcut: does the same
```

Exercise 4: Find the 5 biggest folders inside /var

```
sudo du -h --max-depth=1 /var 2>/dev/null | sort -rh | head -5
df -h /var                     # how full is the disk?
```

10. Quick Cheat Sheet

Command	What it does
`grep 'text' file`	Show lines containing text
`grep -i / -v / -n / -c`	Ignore case / invert / line numbers / count
`grep -r 'text' folder`	Search inside all files of a folder
`cat file`	Show a file
`sort file / sort -n / sort -r`	Alphabetical / numeric / reverse
`sort -k2 -n file`	Sort by 2nd column, numerically
`sort file | uniq`	Remove duplicate lines

Command	What it does
`sort file | uniq -c`	Count each line
`find . -name "*.txt"`	Find files by name
`find . -type f / -type d`	Only files / only folders
`find . -size +1M`	Files bigger than 1 MB
`find . -mtime -7`	Modified in the last 7 days
`df -h`	Free and used space of all disks
`df -hT`	Same, with file system type
`du -sh folder`	Total size of one folder
`du -sh *`	Size of every item in this folder
`du -h --max-depth=1 | sort -rh`	Biggest sub-folders first

Practice questions

- What is the difference between **grep 'error'** and **grep -i 'error'**?
- Why does **sort** put 10 before 2, and how do you fix it?
- Why must we run **sort** before **uniq**?
- Write a command to find files bigger than 100 KB in /home.
- What do **-mtime -7** and **-mtime +7** mean?
- Write a command that shows how many times each word appears in words.txt, most frequent first.
- What is the difference between **df -h** and **du -sh**?
- Why is **df -sh** wrong? What did the student probably mean?
- Write a command to list the 5 biggest sub-folders of /var.

Hint: Answer to the last question: `sort words.txt | uniq -c | sort -rn`

Summary: Conclusion: grep finds text, sort and uniq organise it, find locates files, and df and du show where your disk space goes. Joined with pipes, they handle large amounts of data quickly.